

《深度学习》



复习

<https://nndl.github.io/>
<https://playground.tensorflow.org/>

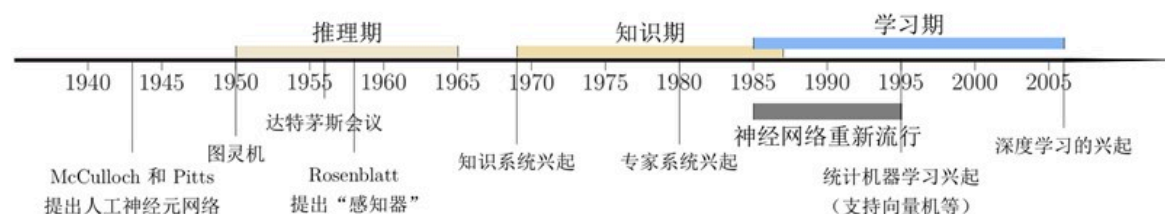
复习要点

- (1) 认真复习，多花几节课的时间看一遍，会有很大收获
- (2) 遵守考试纪律，不作弊
- (3) 认真作答，不留空题
- (4) 作业和实验报告按时完成并提交，必须交到QQ群作业中

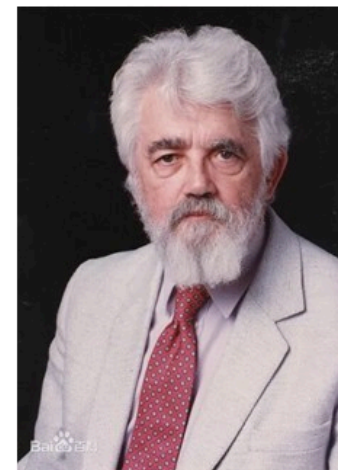
课程大纲

- ▶ 绪论
- ▶ 基础网络模型
 - ▶ 前馈神经网络
 - ▶ 卷积神经网络
 - ▶ 循环神经网络
 - ▶ 网络优化与正则化
 - ▶ 注意力机制与外部记忆
 - ▶ 无监督学习
 - ▶ 模型独立的学习方式

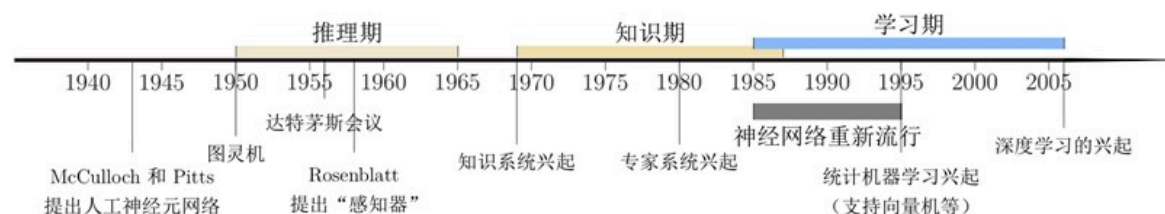
发展历史



人工智能是计算机科学的一个分支，主要研究、开发用于模拟、延伸和扩展人类智能的理论、方法、技术及应用系统等。和很多其他学科不同，人工智能这个学科的诞生有着明确的标志性事件，就是1956年的达特茅斯（Dartmouth）会议。在这次会议上，“人工智能”被提出并作为本研究领域的名称。同时，人工智能研究的使命也得以确定。John McCarthy提出了人工智能的定义：人工智能就是要让机器的行为看起来就像是人所表现出的智能行为一样。



人工智能的发展历史---知识期



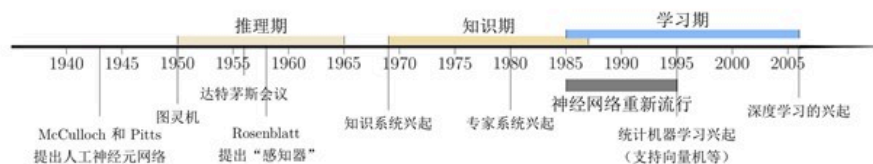
在60年代中后期的时候，人工智能程序在推理能力上已经达到或者说接近了人类的巅峰水平。但是系统有没有聪明起来呢？大家发现还没有，所以我们的前辈就开始反思了。有了逻辑推理能力好像还不是万能的。即便对数学家来说，能够证明复杂的定理，不光是因为他有逻辑推理能力，还需要有很多知识。

在这一时期，出现了各种各样的专家系统（Expert System），并在特定的专业领域取得了很多成果。专家系统可以简单理解为“知识库+推理机”，是一类具有专门知识和经验和计算机智能程序系统。专家系统一般采用知识表示和知识推理等技术来完成通常由领域专家才能解决的复杂问题，因此专家系统也被称为基于知识的系统。

一个专家系统必须具备三要素：

- 1) 领域专家知识；
- 2) 模拟专家思维；
- 3) 达到专家级的水平。

人工智能的发展历史---学习期



怎么破?

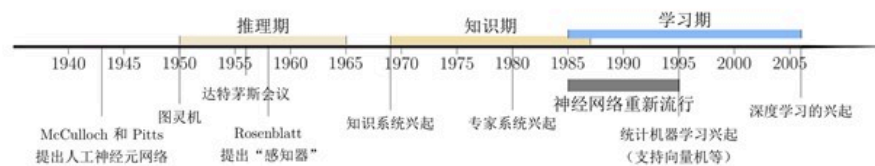
问：知识从哪儿来的？ 答：学来的！

为什么机器学习叫机器学习？

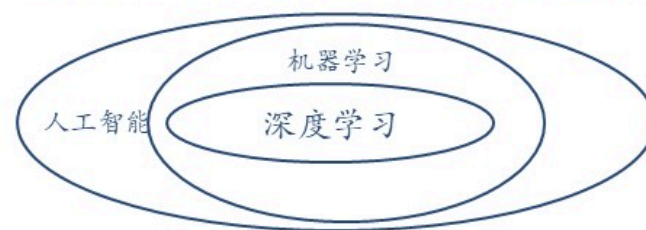
人工智能自然就进入了第三个阶段。这个阶段大概从1990年代中期一直到今天。基本的出发点是希望让系统自己来学知识。这就使得机器学习这个领域现在变成了人工智能的主流核心领域。

机器学习是人工智能的主流。深度学习又是机器学习的主流。

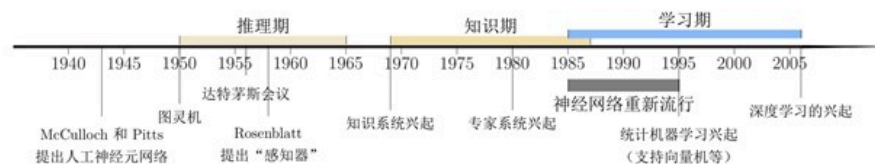
人工智能的发展历史---学习期



机器学习是人工智能的主流。深度学习又是机器学习的主流。



人工智能的发展历史---学习期



研究者开始将研究重点转向让计算机从数据中自己学习。“学习”本身也是一种智能行为。从人工智能的萌芽时期开始，就有一些研究者尝试让机器来学习自动学习，即机器学习 (Machine Learning, ML)。机器学习的主要目的是设计和分析一些学习算法，让计算机可以从数据（经验）中自动分析并获得规律，之后利用学习到的规律对未知数据进行预测，从而帮助人们完成一些特定任务，提高开发效率。在人工智能领域，机器学习从一开始就是一个重要的研究方向。但到1980年后，机器学习因其在很多领域的出色表现，才逐渐称为了热门学科。

1.3 表示学习

► 为了提高机器学习系统的准确率，我们就需要将输入信息转换为有效的特征，或者更一般性地称为表示 (Representation) 。如果有一种算法可以自动地学习出有效的特征，并提高最终机器学习模型的性能，那么这种学习就可以叫作表示学习 (Representation Learning) 。

什么是好的数据表示?

- ▶ “好的表示” 是一个非常主观的概念，没有一个明确的标准。
- ▶ 但一般而言，一个好的表示具有以下几个优点：
 - ▶ 应该具有很强的表示能力。
 - ▶ 应该使后续的学习任务变得简单。
 - ▶ 应该具有一般性，是任务或领域独立的。

内容

▶ 神经网络

- ▶ 神经元

- ▶ 网络结构

▶ 前馈神经网络

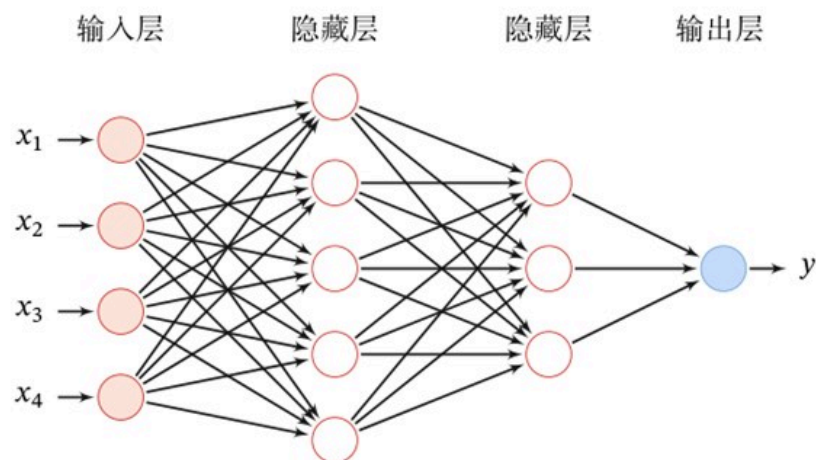
- ▶ 参数学习

- ▶ 计算图与自动微分

- ▶ 优化问题

前馈网络

给定一个前馈神经网络，用下面的记号来描述这样网络：



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$W^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$b^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$z^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$a^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

信息传递过程

► 前馈神经网络通过下面公式进行信息传播。

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

► 前馈计算：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

神经元

$$z = \sum_{d=1}^D w_d x_d + b$$

$$= \mathbf{w}^T \mathbf{x} + b,$$

$$a = f(z),$$

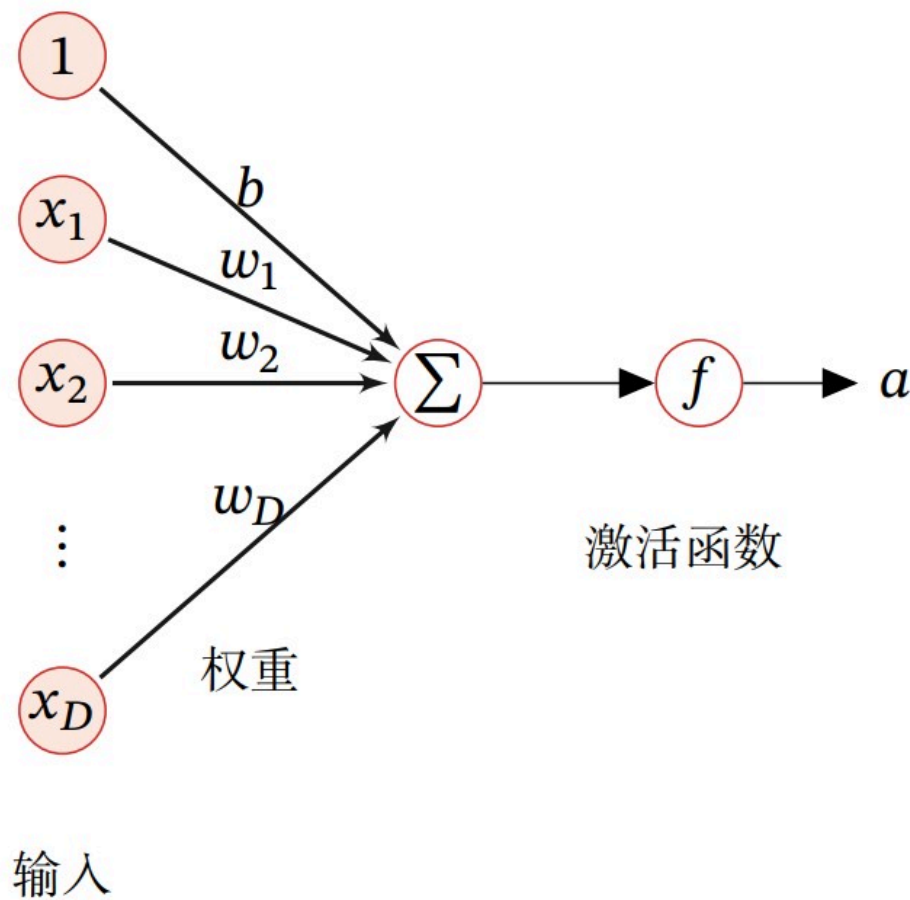


图 4.1 典型的神经元结构

激活函数

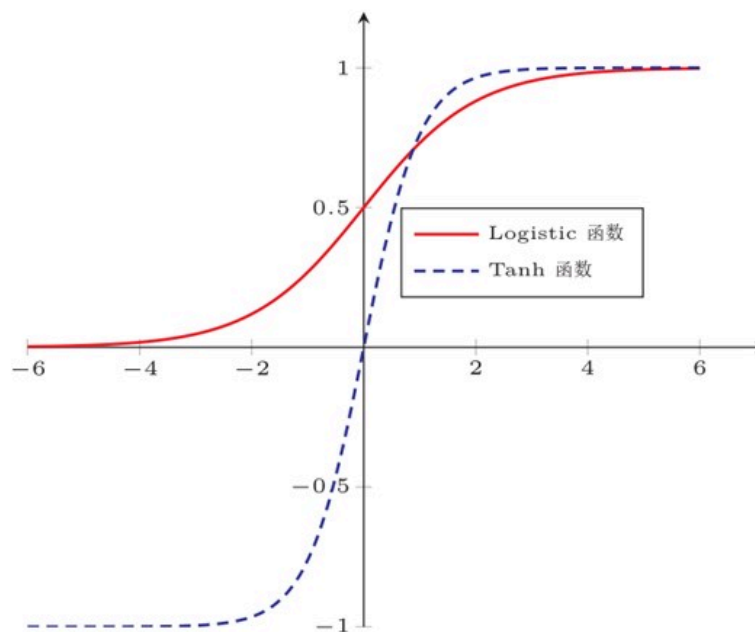
Sigmoid 型函数是指一类 S 型曲线函数，为两端饱和函数。常用的 Sigmoid 型函数有 **Logistic** 函数和 **Tanh** 函数。

Logistic 函数
$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Logistic 函数可以看成是一个“挤压”函数，把一个实数域的输入“挤压”到 $(0, 1)$ 。当输入值在 0 附近时，Sigmoid 型函数近似为线性函数；当输入值靠近两端时，对输入进行抑制。输入越小，越接近于 0；输入越大，越接近于 1。这样的特点也和生物神经元类似，对一些输入会产生兴奋（输出为 1），对另一些输入产生抑制（输出为 0）。和感知器使用的阶跃激活函数相比，Logistic 函数是连续可导的，其数学性质更好。

激活函数

Tanh 函数 $\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$



反向传播算法

使用该算法进行神经网络训练的过程

在计算出每一层的误差项之后，我们就可以得到每一层参数的梯度。因此，使用误差反向传播算法的前馈神经网络训练过程可以分为以下三步：

- (1) 前馈计算每一层的净输入 $\mathbf{z}^{(l)}$ 和激活值 $\mathbf{a}^{(l)}$ ，直到最后一层；
- (2) 反向传播计算每一层的误差项 $\delta^{(l)}$ ；
- (3) 计算每一层参数的偏导数，并更新参数。

反向传播算法

使用该算法进行神经网络训练的过程

算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $M_l, 1 \leq l \leq L$.

```
1 随机初始化  $\mathbf{W}, \mathbf{b}$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;  
7     反向传播计算每一层的误差  $\delta^{(l)}$ ; // 公式 (4.63)  
      // 计算每一层参数的导数  
8      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, y^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$ ; // 公式 (4.68)  
9      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, y^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ; // 公式 (4.69)  
      // 更新参数  
10     $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$ ;  
11     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;  
12  end  
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;  
输出:  $\mathbf{W}, \mathbf{b}$ 
```

20

反向传播算法

假设采用随机梯度下降(stochastic gradient descent, SGD)进行神经网络参数学习, 给定一个样本 $(\mathbf{x}, \mathbf{y})$, 将其输入到神经网络模型中, 得到网络输出为 $\hat{\mathbf{y}}$. 假设损失函数为 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$, 要进行参数学习就需要计算损失函数关于每个参数的导数.

不失一般性, 对第 l 层中的参数 $\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 计算偏导数. 因为 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}}$ 的计算涉及向量对矩阵的微分, 十分繁琐, 因此我们先计算 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于参数矩阵中每个元素的偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}}$. 根据链式法则,

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} &= \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}, \\ \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} &= \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}.\end{aligned}$$

← 误差项

我们只需要计算三个偏导数 $\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}$, $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$ 和 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$

反向传播算法 第 l 层的净输入 $\mathbf{z}^{(l)}$ 对第 l 层的神经元权重 $w_{ij}^{(l)}$ 的偏导数 $\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}$

(1) 计算偏导数 $\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}}$ 因 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, 偏导数

$$\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} = \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{M_l}^{(l)}}{\partial w_{ij}^{(l)}} \right] \quad (4.51)$$

$$= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \quad (4.52)$$

$$= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \quad (4.53)$$

$$\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{1 \times M_l}, \quad (4.54)$$

其中 $\mathbf{w}_{i:}^{(l)}$ 为权重矩阵 $\mathbf{W}^{(l)}$ 的第 i 行, $\mathbb{I}_i(a_j^{(l-1)})$ 表示第 i 个元素为 $a_j^{(l-1)}$, 其余为 0 的行向量.

反向传播算法 第 l 层的净输入 $\mathbf{z}^{(l)}$ 对第 l 层的神经元偏置 $\mathbf{b}^{(l)}$ 的偏导数 $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$

(2) 计算偏导数 $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$ 因为 $\mathbf{z}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 的函数关系为 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, 因此偏导数

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{M_l} \in \mathbb{R}^{M_l \times M_l}, \quad (4.55)$$

为 $M_l \times M_l$ 的单位矩阵.

反向传播算法 损失函数 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 对第 l 层的神经网络净输入 $\mathbf{z}^{(l)}$ 的偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$

(3) 计算偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$ 偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$ 表示第 l 层神经元对最终损失的影响, 也反映了最终损失对第 l 层神经元的敏感程度, 因此一般称为第 l 层神经元的**误差项**, 用 $\delta^{(l)}$ 来表示.

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{M_l}. \quad (4.56)$$

误差项 $\delta^{(l)}$ 也间接反映了不同神经元对网络能力的贡献程度, 从而比较好地解决了**贡献度分配问题** (Credit Assignment Problem, CAP).

根据 $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)}\mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$, 有

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (\mathbf{W}^{(l+1)})^\top \in \mathbb{R}^{M_l \times M_{l+1}}. \quad (4.57)$$

反向传播算法 损失函数 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 对第 l 层的神经网络净输入 $\mathbf{z}^{(l)}$ 的偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$

根据 $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$, 其中 $f_l(\cdot)$ 为按位计算的函数, 因此有

$$\frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \quad (4.58)$$

$$= \text{diag}(f'_l(\mathbf{z}^{(l)})) \in \mathbb{R}^{M_l \times M_l}. \quad (4.59)$$

因此, 根据链式法则, 第 l 层的误差项为

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \quad (4.60)$$

$$= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \quad (4.61)$$

$$= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (\mathbf{W}^{(l+1)})^\top \cdot \delta^{(l+1)} \quad (4.62)$$

$$= f'_l(\mathbf{z}^{(l)}) \odot ((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}) \in \mathbb{R}^{M_l}, \quad (4.63)$$

※神经网络学习体系及符号※

20

反向传播算法 损失函数 $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 对第 l 层的神经网络净输入 $\mathbf{z}^{(l)}$ 的偏导数 $\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$

$$\delta^{(l)} = f'_l(\mathbf{z}^{(l)}) \odot \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \right) \in \mathbb{R}^{M_l}, \quad (4.63)$$

从公式(4.63)可以看出，第 l 层的误差项可以通过第 $l+1$ 层的误差项计算得到，这就是误差的**反向传播**（BackPropagation, BP）。

反向传播算法的含义是：

- 第 l 层的一个神经元的误差项（或敏感性）是所有与该神经元相连的第 $l+1$ 层的神经元的误差项的权重和。然后，再乘上该神经元激活函数的梯度。

《深度学习》



卷积神经网络

<https://nndl.github.io/>

用卷积来代替全连接

根据卷积的定义，卷积层有两个很重要的性质

局部连接：

在卷积层（假设是第 l 层）中的每一个神经元都只和前一层（第 $l-1$ 层）中某个局部窗口内的神经元相连，构成一个局部连接网络。如图5.5b所示，卷积层和前一层之间的连接数大大减少，由原来的 $M_l \times M_{l-1}$ 个连接变为 $M_l \times K$ 个连接， K 为卷积核大小。

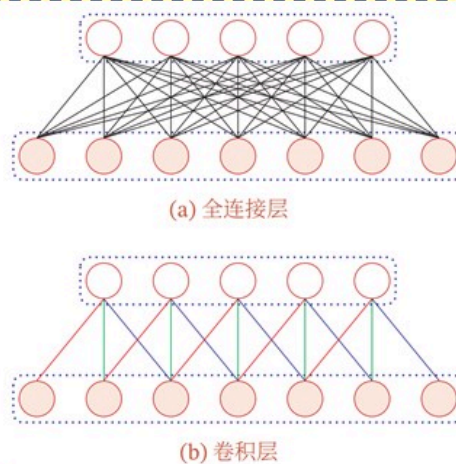


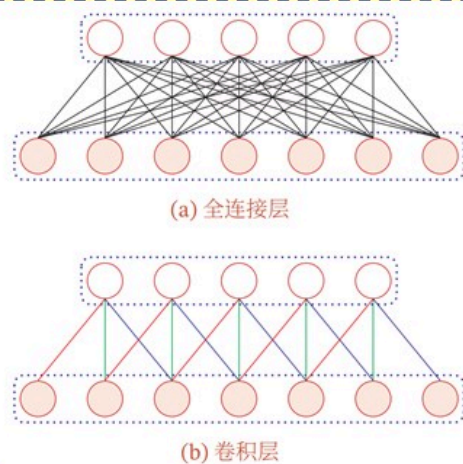
图 5.5 全连接层和卷积层对比

用卷积来代替全连接

根据卷积的定义，卷积层有两个很重要的性质

权重共享：

从公式(5.22)可以看出，作为参数的卷积核 $w^{(l)}$ 对于第 l 层的所有神经元都是相同的。如图5.5b中，所有的同颜色连接上的权重是相同的。权重共享可以理解为一个卷积核只捕捉输入数据中的一种特定的局部特征。因此，如果要提取多种特征就需要使用多个不同的卷积核



$$z^{(l)} = w^{(l)} \otimes a^{(l-1)} + b^{(l)}, \quad (5.22)$$

图 5.5 全连接层和卷积层对比

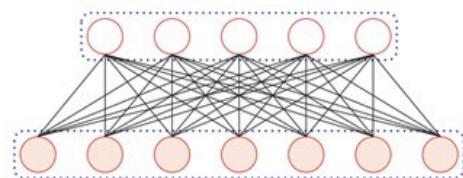
《神经网络与深度学习》

30

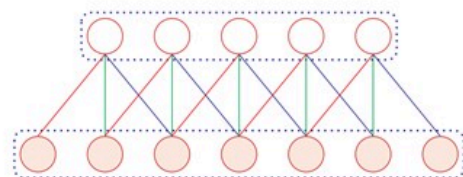
用卷积来代替全连接

卷积神经元数量:

由于局部连接和权重共享, 卷积层的参数只有一个 K 维的权重 $\mathbf{w}^{(l)}$ 和1维的偏置 $b^{(l)}$, 共 $K+1$ 个参数. **参数个数和神经元的数量无关.** 此外, 第 l 层的神经元个数不是任意选择的, 而是满足 $M_l = M_{l-1} - K + 1$.



(a) 全连接层



(b) 卷积层

图 5.5 全连接层和卷积层对比

$$\mathbf{z}^{(l)} = \mathbf{w}^{(l)} \otimes \mathbf{a}^{(l-1)} + b^{(l)}, \quad (5.22)$$

卷积层

特征映射 (Feature Map) 为一幅图像 (或其他特征映射) 在经过卷积提取到的特征, 每个特征映射可以作为一类抽取的图像特征. 为了提高卷积网络的表示能力, 可以在每一层使用多个不同的特征映射, 以更好地表示图像的特征。

为了计算输出特征映射 Y_p , 用卷积核 $W_{p,1}, W_{p,2}, \dots, W_{p,D}$ 分别对输入特征映射 $X_1, X_2, \dots, X_D$ 进行卷积, 然后将卷积结果相加, 并加上一个标量偏置 b_p , 得到卷积层的净输入 Z^p , 再经过非线性激活函数后得到 Y^p

$$Z^p = W^p \otimes X + b^p = \sum_{d=1}^D W^{p,d} \otimes X^d + b^p, \quad (5.23)$$

$$Y^p = f(Z^p). \quad (5.24)$$

其中 $W_p \in \mathbb{R}^{U \times V \times D}$ 为三维卷积核, $f(\cdot)$ 为非线性激活函数, 一般用ReLU函数

卷积层

特征映射 (Feature Map) 为一幅图像 (或其他特征映射) 在经过卷积提取到的特征, 每个特征映射可以作为一类抽取的图像特征. 为了提高卷积网络的表示能力, 可以在每一层使用多个不同的特征映射, 以更好地表示图像的特征。

整个计算过程如图5.7所示. 如果希望卷积层输出 P 个特征映射, 可以将上述计算过程重复 P 次, 得到 P 个输出特征映射 $Y_1, Y_2, \dots, Y_P$.

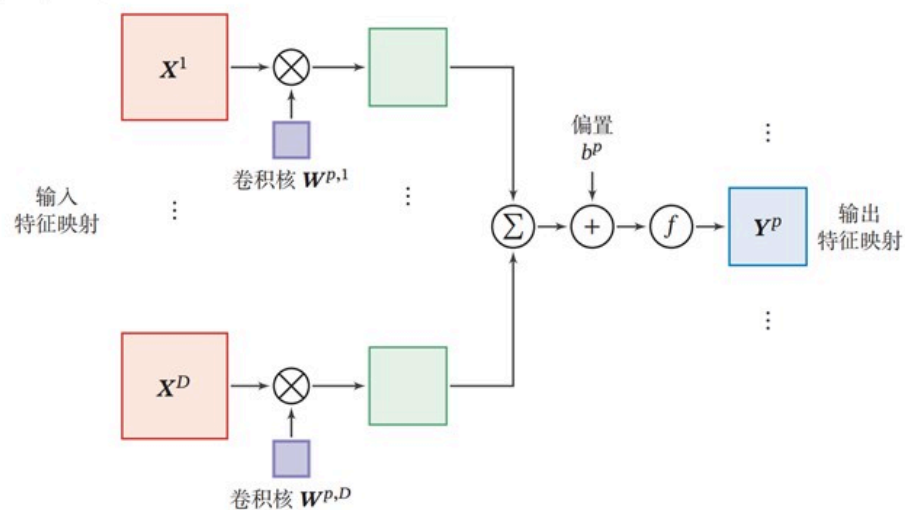


图 5.7 卷积层中从输入特征映射组 X 到输出特征映射 Y^P 的计算示例

卷积层

特征映射 (Feature Map) 为一幅图像 (或其他特征映射) 在经过卷积提取到的特征, 每个特征映射可以作为一类抽取的图像特征. 为了提高卷积网络的表示能力, 可以在每一层使用多个不同的特征映射, 以更好地表示图像的特征。

在输入为 $X \in \mathbb{R}^{M \times N \times D}$, 输出为 $Y \in \mathbb{R}^{M' \times N' \times P}$ 的卷积层中, 每一个输出特征映射都需要 D 个卷积核以及一个偏置. 假设每个卷积核的大小为 $U \times V$, 那么共需要 $P \times D \times (U \times V) + P$ 个参数.

?

汇聚层（池化层，Pooling Layer）

汇聚层（Pooling Layer）也叫子采样层（Subsampling Layer），其作用是进行特征选择，降低特征数量，从而减少参数数量

卷积层虽然可以显著减少网络中连接的数量，但特征映射组中的神经元个数并没有显著减少。如果后面接一个分类器，分类器的输入维数依然很高，很容易出现过拟合。为了解决这个问题，可以在卷积层之后加上一个汇聚层，从而降低特征维数，避免过拟合

假设汇聚层的输入特征映射组为 $X \in \mathbb{R}^{M \times N \times D}$ ，对于其中每一个特征映射实现 $X^d \in \mathbb{R}^{M \times N}$, $1 \leq d \leq D$ ，将其划分为很多区域 $R_{m,n}^d$, $1 \leq m \leq M', 1 \leq n \leq N'$ ，这些区域可以重叠，也可以不重叠。汇聚（Pooling）是指对每个区域进行下采样（Down Sampling）得到一个值，作为这个区域的概括。

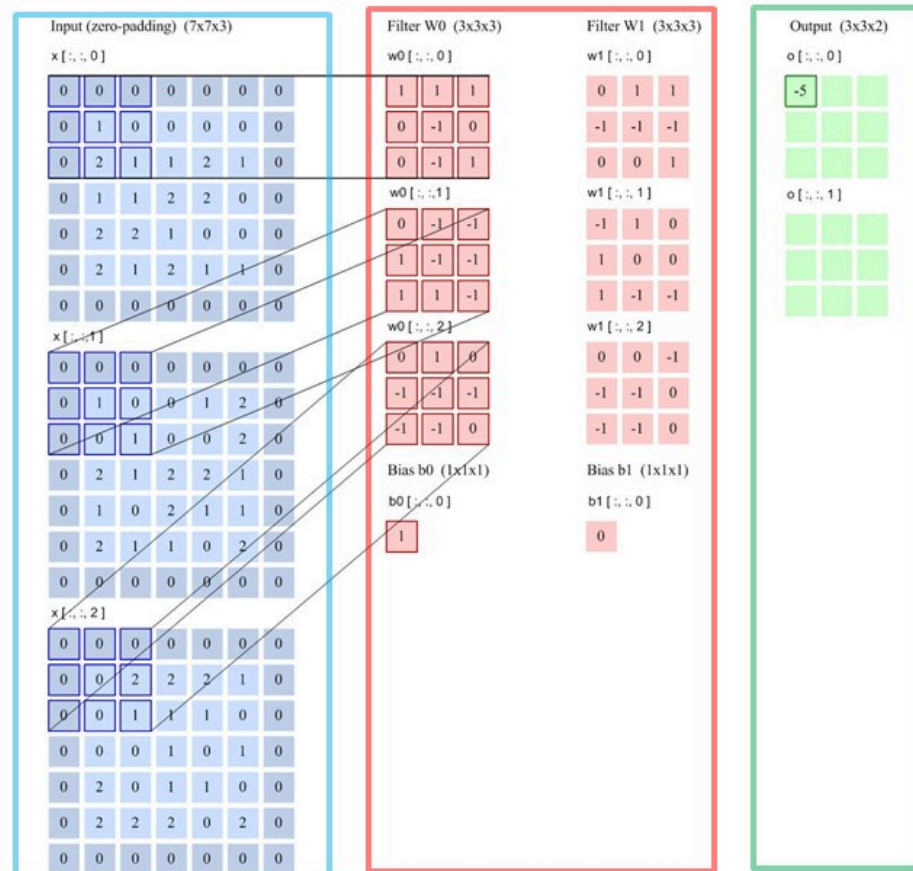
汇聚层（池化层，Pooling Layer）

(1) 最大汇聚（Maximum Pooling或Max Pooling）：对于一个区域 $R_{m,n}^d$ ，选择这个区域内所有神经元的最大活性值作为这个区域的表示，即

$$y_{m,n}^d = \max_{i \in R_{m,n}^d} x_i, \quad (5.25)$$

(2) 平均汇聚（Mean Pooling）：一般是取区域内所有神经元活性值的平均值，即

$$y_{m,n}^d = \frac{1}{|R_{m,n}^d|} \sum_{i \in R_{m,n}^d} x_i. \quad (5.26)$$



步长2
filter 3*3
filter个数6
零填充 1

《深度学习》Deep Learning



Recurrent Neural Network (RNN)

杭州电子科技大学

简单循环网络 (Simple Recurrent Network, SRN)

令向量 $\mathbf{x}_t \in \mathbb{R}^M$ 表示在时刻 t 时网络的输入, $\mathbf{h}_t \in \mathbb{R}^D$ 表示隐藏层状态 (即隐藏层神经元活性值), 则 $\mathbf{h}_t$ 不仅和当前时刻的输入 $\mathbf{x}_t$ 相关, 也和上一个时刻的隐藏层状态 $\mathbf{h}_{t-1}$ 相关. 简单循环网络在时刻 t 的更新公式为

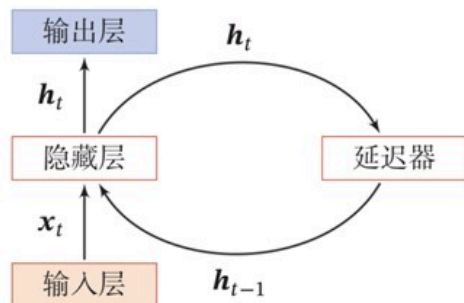
$$\mathbf{z}_t = \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}, \quad (6.5)$$

$$\mathbf{h}_t = f(\mathbf{z}_t), \quad (6.6)$$

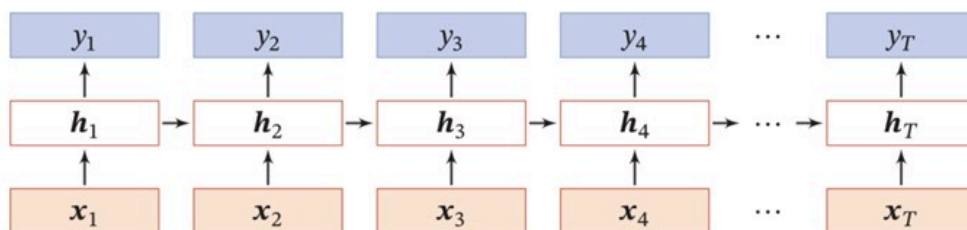
$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}). \quad (6.7)$$

其中 $\mathbf{z}_t$ 为隐藏层的净输入, $\mathbf{U} \in \mathbb{R}^{D \times D}$ 为状态-状态权重矩阵, $\mathbf{W} \in \mathbb{R}^{D \times M}$ 为状态-输入权重矩阵, $\mathbf{b} \in \mathbb{R}^D$ 为偏置向量, $f(\cdot)$ 是非线性激活函数, 通常为 Logistic 函数或 Tanh 函数. 公式(6.5)和公式(6.6)也经常直接写为

如果我们把每个时刻的状态都看作前馈神经网络的一层, 循环神经网络可以看作在时间维度上权值共享的神经网络.

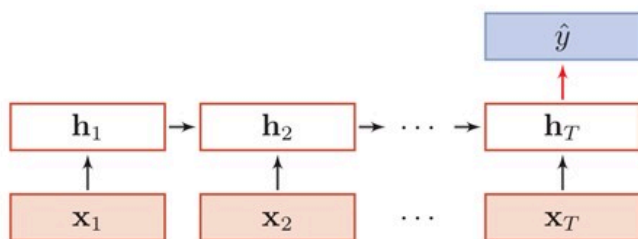


按时间展开

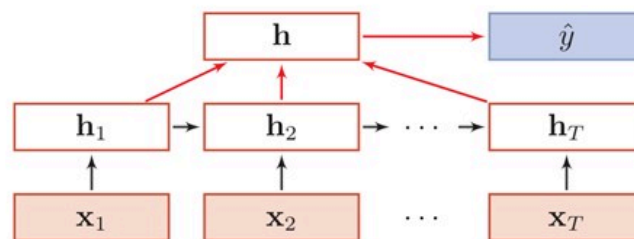


序列到类别模式

- 序列到类别：序列到类别模式主要用于序列数据的分类问题：输入为序列，输出为类别。比如在文本分类中，输入数据为单词的序列，输出为该文本的类别



(a) 正常模式



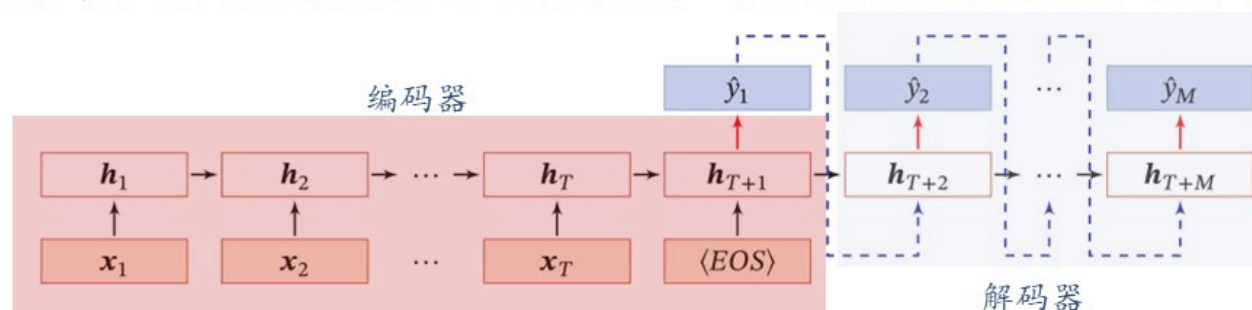
(b) 按时间进行平均采样模式

$$\text{正常模式: } \hat{y} = g(\mathbf{h}_T), \quad (6.22)$$

$$\text{按时间进行平均采样模式: } \hat{y} = g\left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t\right). \quad (6.23)$$

异步的序列到序列模式

异步的序列到序列模式也称为编码器-解码器（**Encoder-Decoder**）模型，即输入序列和输出序列不需要有严格的对应关系，也不需要保持相同的长度。比如在机器翻译中，输入为源语言的单词序列，输出为目标语言的单词序列。

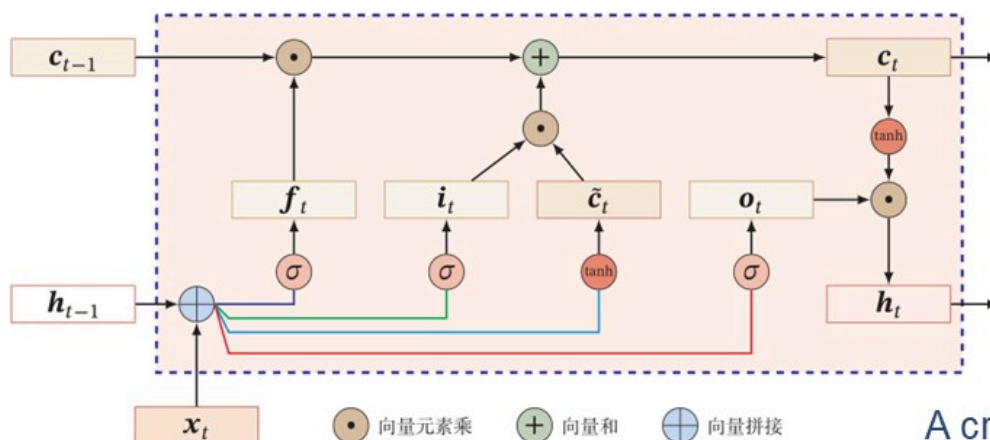


$$h_t = f_1(h_{t-1}, x_t), \quad \forall t \in [1, T] \quad (6.25)$$

$$h_{T+t} = f_2(h_{T+t-1}, \hat{y}_{t-1}), \quad \forall t \in [1, M] \quad (6.26)$$

$$\hat{y}_t = g(h_{T+t}), \quad \forall t \in [1, M] \quad (6.27)$$

Long Short-Term Memory, LSTM



Internal state:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

A crucial addition has been to make the weight on this self-loop conditioned on the context, rather than fixed.

Input Gate $\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i),$

Forget Gate $\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f),$

Output Gate $\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),$

候选状态

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

Deep Learning



网络优化与正则化

<https://nndl.github.io/>

How to improve?

Reference:

1. [An overview of gradient descent optimization algorithms](#)
2. [Optimizing the Gradient Descent](#)



▶ Standard (mini-batch) gradient descent

▶ Learning rate. $\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$

$\Delta\theta_t = -\alpha \mathbf{g}_t$ Gradient direction.

Actual update direction



▶ Learning rate decay

▶ Adagrad

▶ Adadelta

▶ RMSprop

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha \mathbf{g}_t$$

Adam is better choice!

Adam



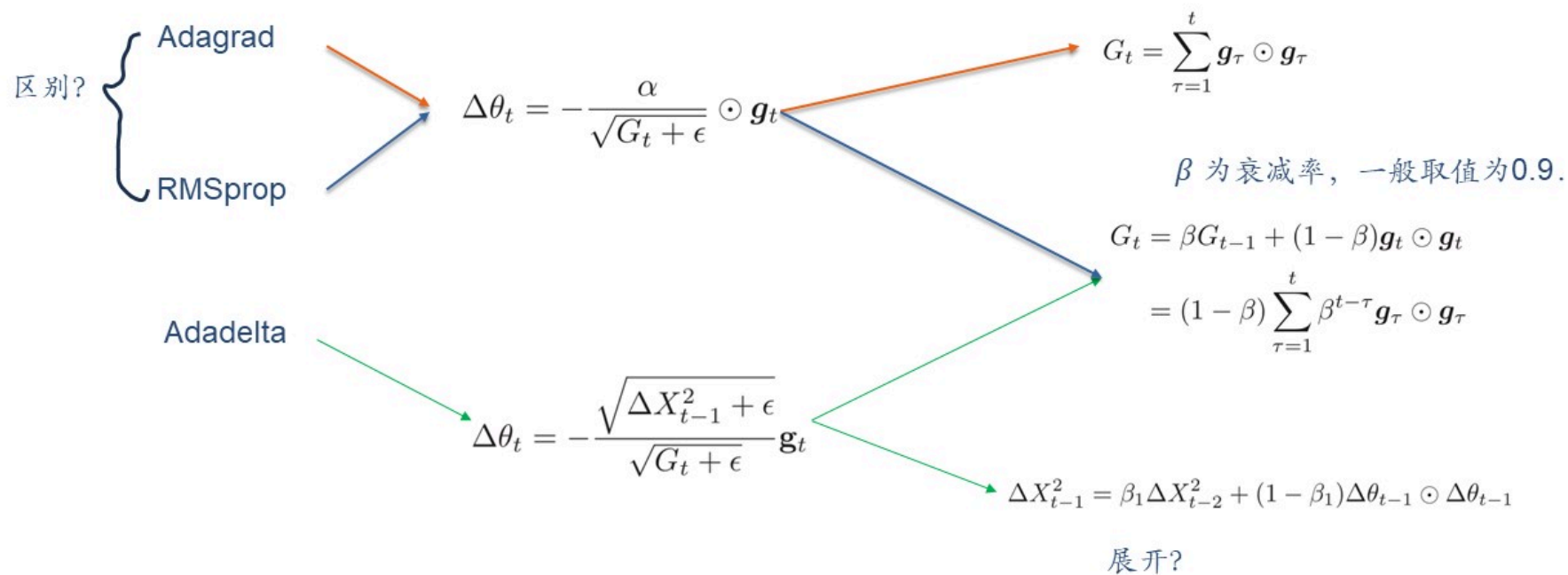
▶ Gradient

▶ Momentum

▶ Calculate the "weighted moving average" of the negative gradient for updating parameters

▶ Gradient clipping

自适应学习率Adadelta



Layer-wise normalization

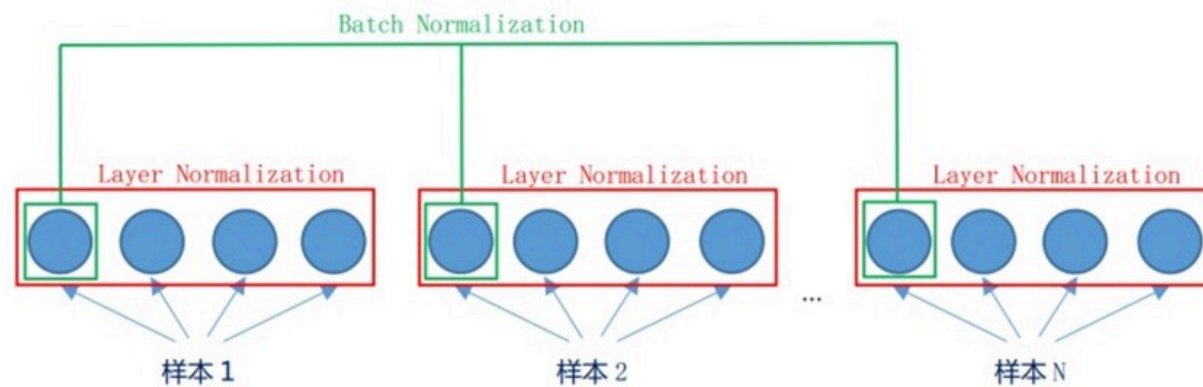
► Purpose

- Improved scale invariance
 - Internal Covariate Shift
- Smoother optimization landscape

► Normalization methods

- Batch Normalization, BN
- Layer Normalization
- Weight Normalization
- Local Response Normalization, LRN

Batch Normalization vs. Layer Normalization



Reconsidering generalization

► Neural network

- Over-parameterization
- Strong fitting capability

↓
~~Poor generalization~~



Zhang C, Bengio S, Hardt M, et al. Understanding deep learning requires rethinking generalization[J]. arXiv preprint arXiv:1611.03530, 2016.

Regularization

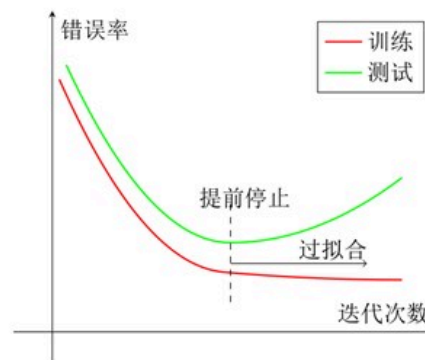
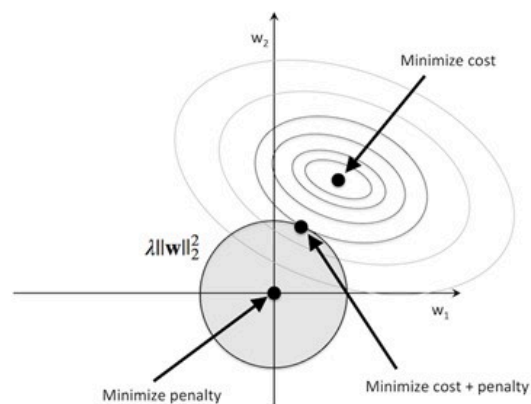
All methods that harm optimization are forms of regularization.

Adding optimization constraints

Disturbing the optimization process

L1/L2 constraints, data augmentation

Weight decay, stochastic gradient descent, early stopping



Regularization

► Methods to Improve Neural Network Generalization Ability:

- early stop
- Weight Decay
- SGD
- Dropout
- Data Augmentation
- ℓ_1 and ℓ_2 Regularization

Deep Learning

<https://bbycroft.net/llm>



注意力机制和外部注意力

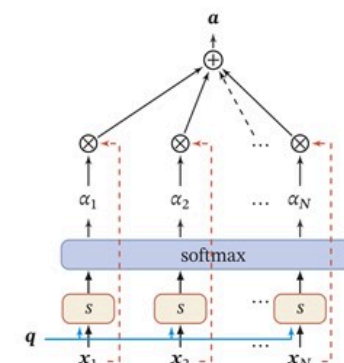
<https://nndl.github.io/>

Attention model

- ▶ Soft attention mechanism
- ▶ The attention mechanism can be divided into two steps
 - ▶ Compute Attention Distribution α ,

$$\begin{aligned}\alpha_n &= p(z = n|X, q) \\ &= \text{softmax}(s(x_n, q)) \\ &= \frac{\exp(s(x_n, q))}{\sum_{j=1}^N \exp(s(x_j, q))}\end{aligned}$$

$s(x_n, q)$ Scoring Function



- ▶ Calculate the weighted average of input information based on α .

$$\begin{aligned}\text{att}(X, q) &= \sum_{n=1}^N \alpha_n x_n, \\ &= \mathbb{E}_{z \sim p(z|X, q)}[x_z]\end{aligned}$$

《神经网络与深度学习》

53

Scoring Function $s(x_n, q)$

Additive Model

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{q}),$$

Dot Product Model

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{q},$$

Scaled Dot Product Model

$$s(\mathbf{x}, \mathbf{q}) = \frac{\mathbf{x}^\top \mathbf{q}}{\sqrt{D}},$$

Bilinear Model

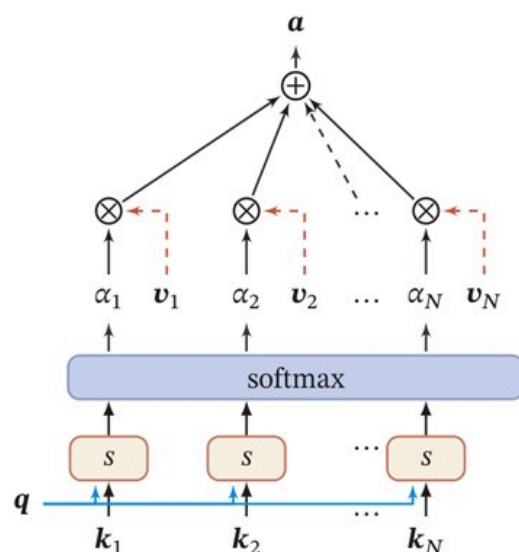
$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{W} \mathbf{q},$$

<https://aistudio.baidu.com/projectdetail/7128926?forkThirdPart=1>

Variants of Attention Mechanism

- ▶ **hard attention**
- ▶ **key-value pair attention**

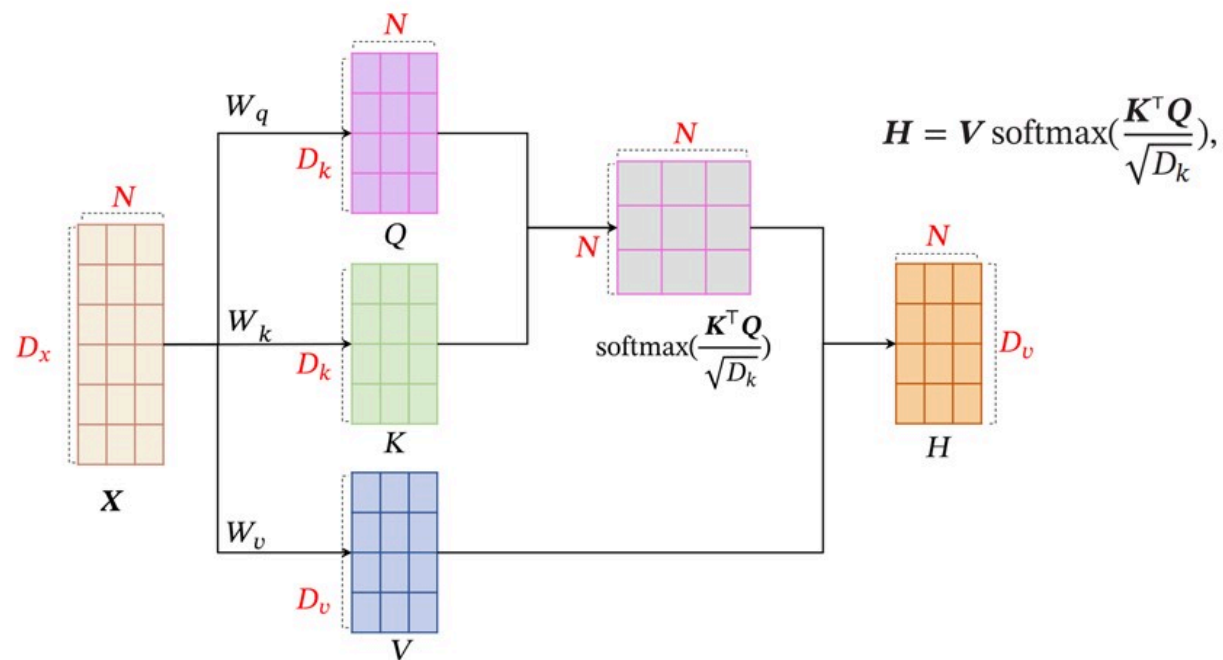
$$\text{att}(\mathbf{X}, \mathbf{q}) = \mathbf{x}_{\hat{n}}, \quad \hat{n} = \arg \max_{n=1}^N \alpha_n.$$



用 $(K, V) = [(k_1, v_1), \dots, (k_N, v_N)]$ 表示 N 个输入信息

$$\begin{aligned} \text{att}((K, V), q) &= \sum_{n=1}^N \alpha_n v_n, \\ &= \sum_{n=1}^N \frac{\exp(s(k_n, q))}{\sum_j \exp(s(k_j, q))} v_n \end{aligned}$$

Query-Key-Value



Self-Attention Model

- ▶ The input sequence is: $X = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{D_x \times N}$
- ▶ First, generate three sequences of vectors

$$Q = W_q X \in \mathbb{R}^{D_k \times N},$$

$$K = W_k X \in \mathbb{R}^{D_k \times N},$$

$$V = W_v X \in \mathbb{R}^{D_v \times N},$$

▶ $\mathbf{h}_n$

$$\mathbf{h}_n = \text{att}((K, V), \mathbf{q}_n)$$

- ▶ If using scaled dot-product as the attention scoring function, the output sequence of vectors can be abbreviated as:

$$H = V \text{softmax}\left(\frac{K^T Q}{\sqrt{D_k}}\right),$$

《深度学习》



无监督学习

<https://nndl.github.io/>

主成份分析 (Principal Component Analysis, PCA)

▶ 一种最常用的数据降维方法，使得在转换后的空间中数据的方差最大。

▶ 样本点 $\mathbf{x}^{(n)}$ 投影之后的表示为

$$\mathbf{z}^{(n)} = \mathbf{w}^T \mathbf{x}^{(n)}$$

▶ 所有样本投影后的方差为

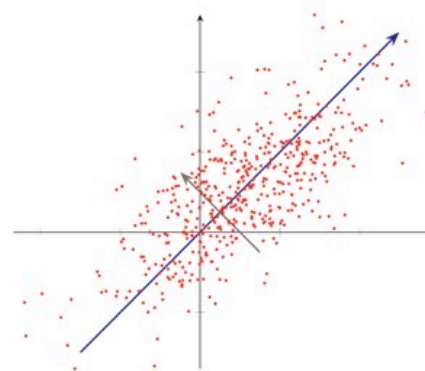
$$\begin{aligned}\sigma(\mathbf{X}; \mathbf{w}) &= \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}^{(n)} - \mathbf{w}^T \bar{\mathbf{x}})^2 \\ &= \frac{1}{N} (\mathbf{w}^T \mathbf{X} - \mathbf{w}^T \bar{\mathbf{X}})(\mathbf{w}^T \mathbf{X} - \mathbf{w}^T \bar{\mathbf{X}})^T \\ &= \mathbf{w}^T \Sigma \mathbf{w},\end{aligned}$$

▶ 目标函数

$$\max_{\mathbf{w}} \mathbf{w}^T \Sigma \mathbf{w} + \lambda(1 - \mathbf{w}^T \mathbf{w})$$

▶ 对目标函数求导并令导数等于 0，可得

$$\Sigma \mathbf{w} = \lambda \mathbf{w}$$



概率密度估计

▶ 参数密度估计 (Parametric Density Estimation)

- ▶ 根据先验知识假设随机变量服从某种分布，然后通过训练样本来估计分布的参数。
- ▶ 估计方法：最大似然估计

$$\log p(\mathcal{D}; \theta) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)}; \theta).$$

▶ 非参数密度估计 (Nonparametric Density Estimation)

- ▶ 不假设数据服从某种分布，通过将样本空间划分为不同的区域并估计每个区域的概率来近似数据的概率密度函数。

参数密度估计一般存在以下问题

▶ 模型选择问题

- ▶ 如何选择数据分布的密度函数?
- ▶ 实际数据的分布往往是非常复杂的，而不是简单的正态分布或多项分布。

▶ 不可观测变量问题

- ▶ 即我们用来训练的样本只包含部分的可观测变量，还有一些非常关键的变量是无法观测的，这导致我们很难准确估计数据的真实分布。

▶ 维度灾难问题

- ▶ 高维数据的参数估计十分困难
- ▶ 随着维度的增加，估计参数所需要的样本数量指数增加。在样本不足时会出现过拟合。

非参数密度估计

- ▶ 对于高维空间中的一个随机向量 $\mathbf{x}$ ，假设其服从一个未知分布 $p(\mathbf{x})$ ，则 $\mathbf{x}$ 落入空间中的小区域 $\mathcal{R}$ 的概率为

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}.$$

- ▶ 给定 N 个训练样本 $D = \{\mathbf{x}^{(n)}\}_{n=1}^N$ ，落入区域 $\mathcal{R}$ 的样本数量 K 服从二项分布

$$P_K = \binom{N}{K} P^K (1-P)^{1-K},$$

- ▶ 当 N 非常大时，我们可以近似认为

$$P \approx \frac{K}{N}$$

- ▶ 假设区域 $\mathcal{R}$ 足够小，其内部的概率密度是相同的，则有

$$P \approx p(\mathbf{x})V$$

- ▶ 结合上述两个公式，得到

$$p(\mathbf{x}) \approx \frac{K}{NV}$$

非参数密度估计

- ▶ 非参数密度估计(直方图方法除外)需要保留整个训练集。
- ▶ 而参数密度估计不需要保留整个训练集，因此在存储和计算上更加高效。

无监督学习

- ▶ 无监督学习是一种十分重要的机器学习方法。广义上讲，监督学习也可以看作一类特殊的无监督学习，即估计条件概率 $p(y|x)$ 。条件概率 $p(y|x)$ 可以通过贝叶斯公式转为估计概率 $p(y)$ 和 $p(x|y)$ ，并通过无监督密度估计来求解；
- ▶ 无监督学习问题主要可以分为聚类、特征学习、密度估计等几种类型。
- ▶ 无监督特征学习是一种十分重要的表示学习方法。当一个监督学习任务的数据比较少时，可以通过大规模的无标注数据，学习到一种有效的数据表示，并有效提高监督学习的性能。
- ▶ 目前，无监督学习并没有像监督学习那样取得广泛的成功，其主要原因在于无监督学习缺少有效的客观评价方法，导致很难衡量一个无监督学习方法的好坏。无监督学习的好坏通常需要代入到下游任务中进行验证。

《深度学习》



模型独立的学习方式

内容

- ▶ 集成学习
- ▶ 协同学习
- ▶ 多任务学习
- ▶ 迁移学习
- ▶ 终身学习
- ▶ 元学习

集成学习

▶三个臭皮匠赛过诸葛亮

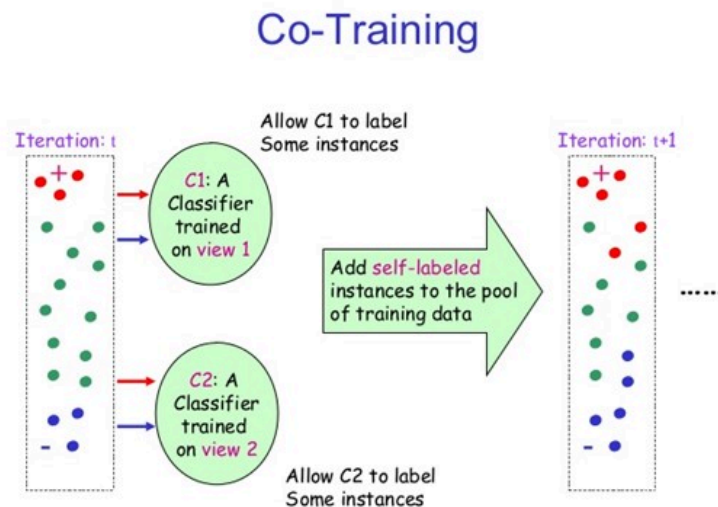
定理 10.1: 对于 M 个不同的模型 $f_1(\mathbf{x}), \dots, f_M(\mathbf{x})$, 平均期望错误为 $\bar{\mathcal{R}}(f)$ 。基于简单投票机制的集成模型 $f^{(c)}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x})$, 其期望错误在 $\frac{1}{M} \bar{\mathcal{R}}(f)$ 和 $\bar{\mathcal{R}}(f)$ 之间。

为了增加模型之间的差异性, 可以采取Bagging和Boosting这两类方法.

协同训练 (Co-Training)

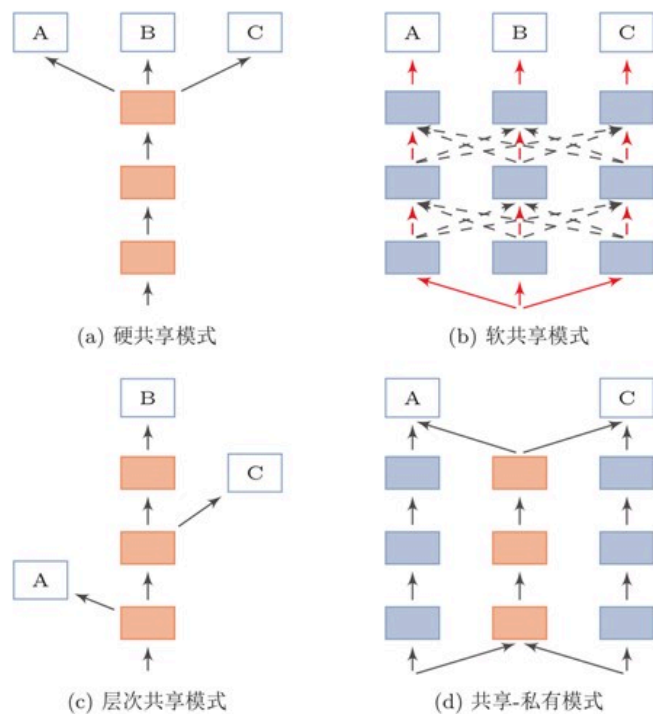
► Multi-View Learning

协同训练 (Co-Training) 是自训练的一种改进方法，通过两个基于不同视角 (view) 的分类器来互相促进



由于不同视角的条件独立性，在不同视角上训练出来的模型就相当于从不同视角来理解问题，具有一定的互补性。协同训练就是利用这种互补性来进行自训练的一种方法。首先在训练集上根据不同视角分别训练两个模型 f_1 和 f_2 ，然后用 f_1 和 f_2 在无标注数据集上进行预测，各选取预测置信度比较高的样本加入训练集，重新训练两个不同视角的模型，并不断重复这个过程。

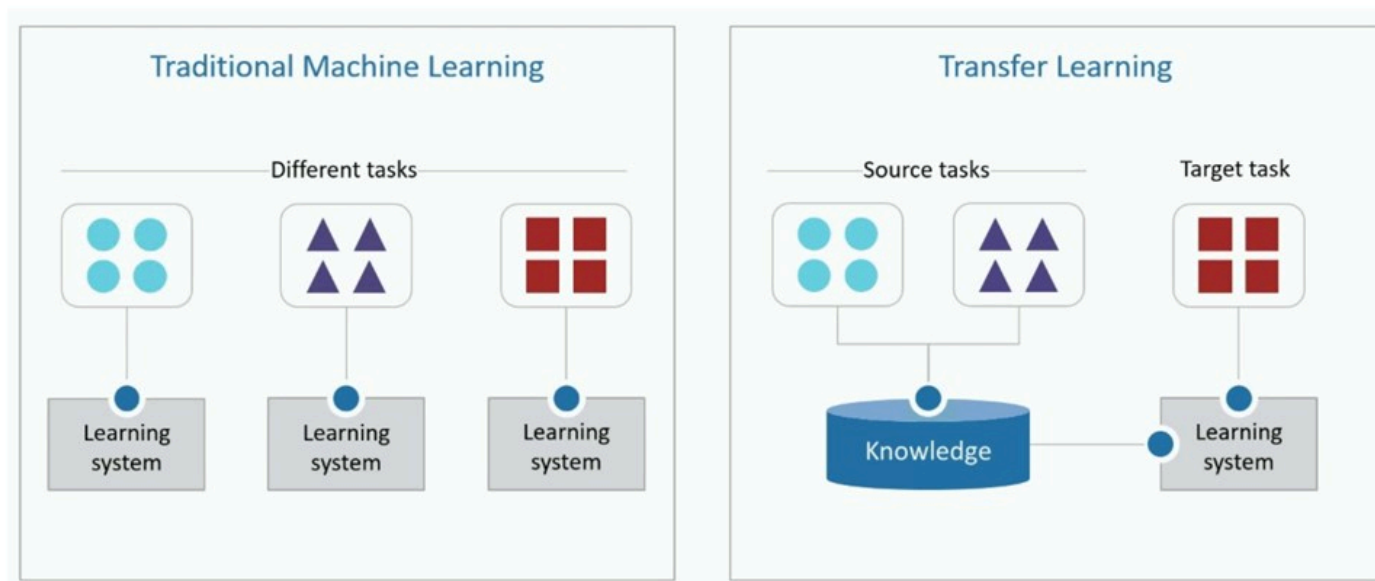
多任务学习 (Multitask Learning)



多任务学习 (Multi-task Learning) 是指同时学习多个相关任务, 让这些任务在学习过程中共享知识, 利用多个任务之间的相关性来改进模型在每个任务上的性能和泛化能力. 多任务学习可以看作一种 **归纳迁移学习** (Inductive Transfer Learning), 即通过利用包含在相关任务中的信息作为归纳偏置 (Inductive Bias) 来提高泛化能力。

迁移学习 (Transfer Learning)

迁移学习是指两个不同领域的知识迁移过程，利用源领域 (Source Domain) DS 中学到的知识来帮助目标领域 (Target Domain) DT 上的学习任务。源领域的训练样本数量一般远大于目标领域。



迁移学习 (Transfer Learning)

迁移学习根据不同的迁移方式又分为两个类型：

- **归纳迁移学习 (Inductive Transfer Learning)**：一般的机器学习都是指归纳学习，即希望在训练数据集上学习到使得期望风险（即真实数据分布上的错误率）最小的模型。归纳迁移学习是指在源领域和任务上学习出一般的规律，然后将这个规律迁移到目标领域和任务上；
- **转导迁移学习 (Transductive Transfer Learning)**：转导学习的目标是学习一种在给定测试集上错误率最小的模型，在训练阶段可以利用测试集的信息。而转导迁移学习是一种从样本到样本的迁移，直接利用源领域和目标领域的样本进行迁移学习。

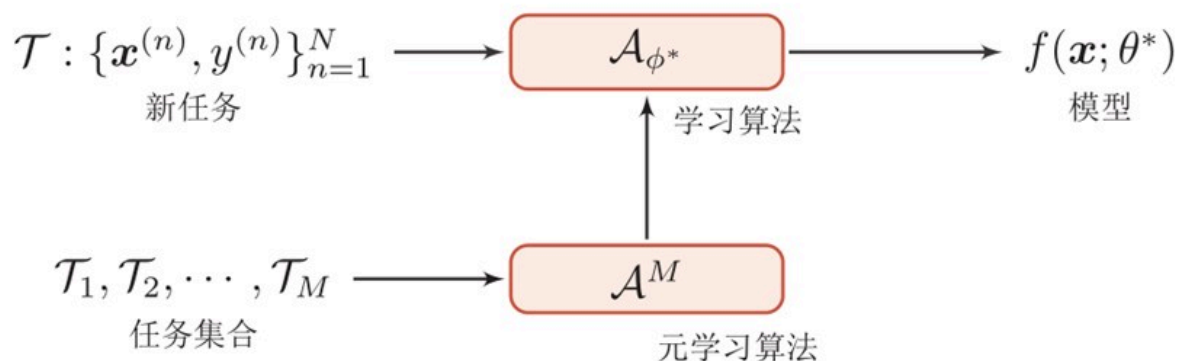
迁移学习 (Transfer Learning)

在归纳迁移学习中，由于源领域的训练数据规模非常大，这些预训练模型通常有比较好的泛化性，其学习到的表示通常也适用于目标任务。归纳迁移学习一般有以下两种迁移方式：

- **基于特征的方式**：将预训练模型的输出或者是中间隐藏层的输出作为特征直接加入到目标任务的学习模型中。目标任务的学习模型可以是一般的浅层分类器（比如支持向量机等）或一个新的神经网络模型；
- **精调的方式**：在目标任务上复用预训练模型的部分组件，并对其参数进行精调（Fine-Tuning）。

元学习 (Meta Learning)

根据没有免费午餐定理，没有一种通用的学习算法可以在所有任务上都有效。因此，当使用机器学习算法实现某个任务时，我们通常需要“就事论事”，根据任务的特点来选择合适的模型、损失函数、优化算法以及超参数。那么，我们是否可以有一套自动方法，根据不同任务来动态地选择合适的模型或动态地调整超参数呢？事实上，人脑中的学习机制就具备这种能力。在面对不同的任务时，人脑的学习机制并不相同。即使面对一个新的任务，人们往往也可以很快找到其学习方式。这种可以动态调整学习方式的能力，称为元学习 (**Meta-Learning**)，也称为学习的学习 (Learning to Learn) [Thrun et al., 2012]。



终身学习

虽然深度学习在很多任务上取得了成功，但是其前提是训练数据和测试数据的分布要相同，一旦训练结束模型就保持固定，不再进行迭代更新。并且，要一个模型同时在很多不同任务上都取得成功依然是一件十分困难的事情。由于不断的知识累积，人脑在学习新的任务时一般不需要太多的标注数据。

终身学习 (Lifelong Learning)，也叫持续学习 (Continuous Learning)，是指像人类一样具有持续不断的学习能力，根据历史任务中学到的经验和知识来帮助学习不断出现的新任务，并且这些经验和知识是持续累积的，不会因为新的任务而忘记旧的知识

